Compiler Construction: Project Phase 2

Due on **Friday March 27th at 11:59 P.M.**

1 Description

Your task for this project is to write a parser for `mC` that will parse the token stream generated by your lexical analyzer. The parser will detect syntax errors for programs that do not meet the specifications of the `mC` grammar. For all syntactically correct programs the parser will construct an Abstract Syntax Tree (AST) representation. The AST will be used later by the semantic analyzer and code generator. Your parser will also need to build a symbol table for keeping track of names within the program.
This assignment comes with a templated parser with a detailed description of what each function and variable should do. Please feel free to implement the project however you wish, so long as the directory structure and names of the files do not change, you are welcome to design your own solution. Please refer to Tables 1–3 for the grading rubric and test cases.

2 Syntax Specification

The grammar for `mC` is attached to this handout. Your first task is to express this grammar using YACC specification rules. You will want to run your specification through YACC to make sure there are no conflicts in the grammar. If there are conflicts, you will need to eliminate them by rewriting the specifications without changing the meaning of the grammar.

3 Abstract Syntax Tree

The AST is a tree representation of the syntax of the input program. You have some flexibility as to how you structure this tree. A reasonable strategy might be to use an `n-ary` tree, where the internal nodes correspond to non-terminals in the grammar and the leaf nodes represent terminals (e.g., `identifier`, `integer constant`). NOTE, this rule does not need to be enforced strictly. In some cases, it may be convenient to have a node in the tree that is neither a terminal nor a non-terminal. In other situations, it may be helpful to add a child node to a terminal. Whatever your implementation strategy, your AST should contain (or have access to) sufficient information so that, by traversing the tree, it is possible to reproduce source code that is equivalent to the original source. This implies that for some terminals, you will need to store the corresponding semantic value (e.g., name for identifiers, numeric value for integer constants, ASCII value for character constants).

4 Printing the Abstract Syntax Tree

Each node is printed on its own line beginning with the root node. Child nodes appear on the lines following their parent node and are indented with an additional two spaces. Print the type of each node and include additional information when applicable. Nodes that should contain additional information include constants, operators, type specifiers, and identifiers. See the sample output provided as an example.

5 Symbol Table

At this phase, the symbol table should contain three types of information for each identifier: `name`, `type`, and `scope`. Like `C`, `mC` has two kinds of scopes for variables: `local` and `global`. Variables declared outside of any function are considered globals, whereas variables (and parameters) declared inside a function `foo` are local to the function `foo`. Each entry in the symbol table should contain the name of the identifier, its scope as the name of the containing function or blank if its scope is global, the data type of the identifier (`int`, `char`, or `void`), and the type of symbol (scalar variable – (`blank`), array variable – "`[]`", or function – "`()`"). The hashkey for a symbol can be calculated by concatenating the identifier's scope and name, using the `djb2` hash function[1] on the resulting string, and taking the result modulo the size of the symbol table.

6 Error Handling

The parser adds detection of two additional types of issues in the source program, errors include:

1. Syntax Error: If the input program does not conform to the grammar of `mC`, the parser reports the line number where the program first deviates from the specified grammar.
2. Use of Undeclared Symbol Warning: As the parser builds the symbol table, it is capable of recognizing usage of variables and functions before they are declared. As this is more of a semantic error than a syntactic one, this issue is only reported as a warning at this stage. Later phases of compilation will likely treat this as an error.
3. Errors Detected by the Scanner: The scanner will recognize and provide line number and column number for the following errors in the input source code:
   (a) Unterminated comment
   (b) Integer with leading zero
   (c) Identifier that begins with a number

7 Implementation and Testing Instructions

- Use YACC or BISON to implement your parser
- The template code contains the following files – `driver.c`, `parser.y`, `scanner.l`, `strtab.c`, `strtab.h`, `tree.c`, `tree.h`, and `Makefile`
- `strtab.h` and `tree.h` contain data structures that you will need to implement the symbol table and the AST
- The `driver.c` file calls the function "`printAST`" that you will need to implement, this function will print the contents of the entire AST
- You need to implement functions in `parser.y`, `strtab.c`, and `tree.c`
- You may use the `scanner.l` from Phase 1
  NOTE*: values which were in* `tokendef.h` *are now generated from* `parser.y`
- A subset of the test cases which will be used are provided for you to test your code, think about edge-cases where your program may fail – testing your code is equally as important as writing the code itself

[1] `http://www.cse.yorku.ca/~oz/hash.html`

- Your program will be tested on Zeus
- The provided `driver.c` accepts input through various means:

input pipe	`cat input.mC | ./obj/mcc`
file redirect	`./obj/mcc < input.mC`
file as argument	`./obj/mcc input.mC`
interactively, use EOF (`ctrl-d`) **for end-of-file**	`./obj/mcc`

8 Writeup

You need to write a short document named `writeup.txt`.
Within `writeup.txt` you should include the following (numbered) sections:

1. Your name and course ID
2. Group members and their respective contributions
3. Provide two examples different from those provided in this document of how you will test the correctness of your code
4. Extra instructions if you do not use the provided `Makefile`

9 Compilation and Testing

- Unless indicated in `writeup.txt`, your program will be compiled using the provided `Makefile` with the commands:
  `make clean ; make`
- Please ensure the latest version of all files are present in your submission

10 Submission

The project may be done individually or as a group of two. Please submit on CANVAS a ZIP archive named "`Project2_<netID1>_<netID2>.zip`" containing your files. Ensure the directory structure is exactly as we have provided. The grading rubric and test cases are provided in Tables 1–3.

11 Sample Input and Output

A set of sample input and output is provided. Don't worry about the indices used in the symbol table, as these may vary depending on the hashing and mapping strategy you use. Some internal nodes may be equivalent, so your AST may have more or fewer nodes than the expected output. Your output should be equivalent to the expected output when ran with the sample input. NOTE: you should be able to reconstruct the input source from your AST.

12 yacc Tutorial

1. Part 01: Tutorial on lex/yacc: `https://www.youtube.com/watch?v=54bo1qaHAfk`
2. Part 02: Tutorial on lex/yacc: `https://www.youtube.com/watch?v=__-wUHG2rfM`

Category	Points
Test Cases (Refer to Table 3)	56
AST	24
Warning Message for Undeclared Variables	5
Identification of Syntax Errors	5
Compile & Run	5
Documentation	5
Writeup	5

Table 1: Rubric for Project Phase 2

Late Amount	Points Deducted
1 Day	10%
2 Days	20%
3 Days	30%
4 Days	40%
5 Days	50%
> 5 Days	100%

Table 2: Late Submission Policy

Category	Points
AddExpr	4
ArithExpr	6
AssgnExpr	4
CondStmt	4
FunDecl	4
FunCall	6
GlobalVars	4
LocalVars	4
LoopStmt	4
MulExpr	6
RelOpExpr	6
ReturnStmt	4

Table 3: Test Cases

13 mC Grammar

⟨*program*⟩ ::= ⟨*declList*⟩

⟨*declList*⟩ ::= ⟨*decl*⟩
| ⟨*declList*⟩ ⟨*decl*⟩

⟨*decl*⟩ ::= ⟨*varDecl*⟩
| ⟨*funDecl*⟩

⟨*varDecl*⟩ :== ⟨*typeSpecifier*⟩ ID [INTCONST] ;
| ⟨*typeSpecifier*⟩ ID ;

⟨*typeSpecifier*⟩ :== int
| char
| void

⟨*funDecl*⟩ :== ⟨*typeSpecifier*⟩ ID (⟨*formalDeclList*⟩) ⟨*funBody*⟩
| ⟨*typeSpecifier*⟩ ID ( ) ⟨*funBody*⟩

⟨*formalDeclList*⟩ :== ⟨*formalDecl*⟩
| ⟨*formalDecl*⟩ , ⟨*formalDeclList*⟩

⟨*formalDecl*⟩ :== ⟨*typeSpecifier*⟩ ID
| ⟨*typeSpecifier*⟩ ID [ ]

⟨*funBody*⟩ :== { ⟨*statementList*⟩ }

⟨*statementList*⟩ :== ε
| ⟨*statement*⟩ ⟨*statementList*⟩

⟨*statement*⟩ :== ;
| ⟨*varDecl*⟩
| ⟨*compoundStmt*⟩
| ⟨*assignStmt*⟩
| ⟨*condStmt*⟩
| ⟨*loopStmt*⟩
| ⟨*returnStmt*⟩

⟨*compoundStmt*⟩ :== { ⟨*statementList*⟩ }

⟨*assignStmt*⟩ :== ⟨*var*⟩ = ⟨*expression*⟩ ;
| ⟨*expression*⟩ ;

⟨*condStmt*⟩ :== if (⟨*expression*⟩) ⟨*statement*⟩
| if (⟨*expression*⟩) ⟨*statement*⟩ else ⟨*statement*⟩

⟨*loopStmt*⟩ :== while (⟨*expression*⟩) ⟨*statement*⟩

⟨*returnStmt*⟩ :== return ;
| return ⟨*expression*⟩ ;

⟨*var*⟩ :== ID
| ID [⟨*addExpr*⟩]

⟨*expression*⟩ :== ⟨*addExpr*⟩
| ⟨*expression*⟩ ⟨*relop*⟩ ⟨*addExpr*⟩

⟨*relop*⟩ :== <=
| <
| >
| >=
| ==
| !=

⟨*addExpr*⟩ :== ⟨*term*⟩
| ⟨*addExpr*⟩ ⟨*addop*⟩ ⟨*term*⟩

⟨*addop*⟩ :== +
| −

⟨*term*⟩ :== ⟨*factor*⟩
| ⟨*term*⟩ ⟨*mulop*⟩ ⟨*factor*⟩

⟨*mulop*⟩ :== *
| /

⟨*factor*⟩ :== (⟨*expression*⟩)
| ⟨*var*⟩
| ⟨*funCallExpr*⟩
| INTCONST
| CHARCONST

⟨*funCallExpr*⟩ :== ID (⟨*argList*⟩)
| ID ( )

⟨*argList*⟩ :== ⟨*expression*⟩
| ⟨*argList*⟩ , ⟨*expression*⟩